

Arquitectura y Diseño de Sistemas 2026

Lineamientos para

Decisiones Arquitectónicas (ADRs)

Entregable 4 · Práctica de Arquitectura y Diseño de Sistemas 2026

Un ADR (Architecture Decision Record) es un documento breve que registra una decisión arquitectónica importante: el contexto que la motivó, las alternativas evaluadas, y las consecuencias de haberla tomado. No documenta decisiones de implementación — solo las que tienen impacto estructural en el sistema.

1. ¿Qué evalúa la cátedra con los ADRs?

La cátedra no busca que elijan la tecnología «correcta» — busca evidencia de que el equipo pensó con criterio:

- Identificaron el problema real que los llevó a decidir.
- Consideraron más de una alternativa viable, no solo la que ya tenían en mente.
- Pueden articular por qué descartaron las otras opciones.
- Entienden los trade-offs de la decisión tomada: qué ganaron y qué perdieron.
- Las decisiones son coherentes entre sí y con el resto del diseño del sistema.

Una decisión sin alternativas consideradas no es una decisión arquitectónica — es un supuesto no cuestionado. El valor del ADR está en demostrar que eligieron con criterio.

2. ¿Cuántos ADRs deben entregar?

Mínimo 4 ADRs por grupo, cubriendo al menos una decisión de cada categoría obligatoria:

Categoría	Ejemplos de decisiones típicas	Mínimo
Arquitectura del backend	Monolito modular vs microservicios, organización en capas, separación de workers	1 ADR
Persistencia de datos	Tipo de BD (relacional/NoSQL), estrategia de caché, modelo de datos	1 ADR
Comunicación entre componentes	REST síncrono vs colas de mensajes, eventos, polling	1 ADR
Infraestructura / Despliegue	Containerización, cloud vs on-premise, estrategia CI/CD	1 ADR (recomendado)
Integración externa	Qué APIs externas se consumen, manejo de fallos, rate limiting	Opcional

3. Estados de un ADR

Cada ADR debe indicar su estado actual. El más común en la entrega es Aceptada.

Estado	Significado
Propuesta	La decisión está siendo evaluada. Aún no es definitiva.
Aceptada	La decisión fue tomada y está vigente. Es el estado más común en la entrega.
Reemplazada	Fue válida pero fue superada por una decisión posterior. Indicar el ADR que la reemplaza.
Deprecada	Ya no aplica porque el contexto cambió.

4. Errores frecuentes a evitar

X Lo que NO hay que hacer	Lo que SÍ hay que hacer
"Elegimos PostgreSQL porque es buena." - Una sola alternativa considerada. - Contexto que describe la solución, no el problema. - Consecuencias solo positivas. - Documentar cada línea de código como ADR.	"Elegimos PostgreSQL porque necesitamos joins complejos y consistencia transaccional." - Al menos 2-3 alternativas reales con ventajas y desventajas. - Contexto que describe el problema de negocio o restricción técnica. - Consecuencias positivas Y trade-offs (qué complejidad añade, qué se pierde). - Solo las decisiones con impacto estructural en el sistema.

5. Plantilla vacía — copiar y completar

Copiar esta estructura para cada ADR. Las secciones son obligatorias.

ID	Título	Estado	Fecha
ADR-XXX	Título que describe la decisión en una línea	Aceptada	DD/MM/AAAA

1. Contexto

Describir el problema de negocio o restricción técnica que obligó a decidir. No describir la solución todavía.

Motivador 1: requerimiento, constraint o atributo de calidad (ej: consistencia, performance, costo, experiencia del equipo).

Motivador 2: otro factor relevante.

2. Alternativas consideradas

Alternativa	Ventaja principal	Desventaja / Motivo de descarte
-------------	-------------------	---------------------------------

Opción 1	Su ventaja en este contexto	Por qué no fue elegida
Opción 2	Su ventaja en este contexto	Por qué no fue elegida
Opción 3 (la elegida)	Su ventaja en este contexto	— (esta fue la elegida)

3. Decisión tomada

Se decide: descripción concisa de la decisión.

Fundamentación: razón 1 que justifica la elección en relación al contexto.

Razón 2 — debe conectar con los motivadores mencionados.

Razón 3 — puede mencionar experiencia del equipo, herramientas disponibles, etc.

4. Consecuencias

Consecuencias positivas	Trade-offs / costos
Beneficio concreto 1 Beneficio concreto 2	Complejidad o limitación que introduce esta decisión Deuda técnica o riesgo que surge

6. Ejemplo resuelto y comentado

El siguiente ADR está completamente resuelto. Los bloques azules explican por qué cada sección está escrita de esa manera. Usarlo como referencia de nivel de detalle y estilo esperado.

Este ejemplo usa un dominio genérico (e-commerce) para que sea comprensible independientemente del sistema de cada grupo. Adaptar el contenido al dominio propio.

ID	Título	Estado	Fecha
ADR-002	Elección de base de datos principal para la capa de persistencia	Aceptada	15/04/2026

El título describe la DECISIÓN, no la tecnología elegida. 'Uso de PostgreSQL' sería incorrecto — revela la solución antes de mostrar el razonamiento.

1. Contexto

El sistema necesita almacenar y consultar entidades con relaciones complejas entre sí: usuarios, pedidos, productos, ítems de pedido y reseñas. Las consultas más frecuentes implican joins entre varias tablas (ej: obtener todos los pedidos de un usuario junto con sus ítems y productos asociados).

Motivador 1 — Consistencia transaccional: el proceso de compra involucra múltiples escrituras (crear pedido, descontar stock, registrar pago) que deben ser atómicas. Si una falla, ninguna debe persistir.

Motivador 2 — Complejidad de las consultas: los reportes del sistema (ventas por período, productos más comprados, historial de usuario) requieren agregaciones sobre datos relacionados.

Motivador 3 — Experiencia del equipo: el equipo tiene experiencia previa con bases de datos

relacionales y con SQL, pero ninguno tiene experiencia productiva con bases de datos no relacionales.

El contexto describe el PROBLEMA, no la solución. Alguien que lea solo esta sección debe entender por qué el equipo tuvo que decidir algo, no qué decidió. Notar que no aparece ninguna tecnología todavía.

2. Alternativas consideradas

Alternativa	Ventaja principal en este contexto	Desventaja / Motivo de descarte
MongoDB (NoSQL documental)	Esquema flexible; buen rendimiento en lecturas simples por documento	No soporta joins nativos ni transacciones multi-documento de forma eficiente. Las consultas de reportes requerirían lógica compleja en la aplicación.
MySQL	Ampliamente conocido, buen soporte de transacciones, comunidad grande	Soporte de JSON y tipos avanzados más limitado que PostgreSQL. Menor flexibilidad para evolucionar el esquema.
PostgreSQL (elegida)	Transacciones ACID completas, soporte nativo de joins complejos, tipos avanzados (JSON, arrays, enums), extensible	— (esta fue la elegida)

Cada alternativa tiene ventaja Y desventaja documentadas. La desventaja de la opción elegida es '—' porque se explica en las consecuencias. Importante: las desventajas de las alternativas descartadas deben conectar con los motivadores del contexto — no son genéricas.

3. Decisión tomada

Se decide: usar PostgreSQL como única base de datos relacional para toda la capa de persistencia transaccional del sistema.

Fundamentación:

1. El modelo de datos del sistema es inherentemente relacional: las entidades centrales (usuarios, pedidos, ítems, productos) están fuertemente vinculadas y las consultas clave requieren joins eficientes que PostgreSQL maneja de forma nativa.
2. Las transacciones ACID de PostgreSQL resuelven directamente el motivador de consistencia: el flujo de compra puede ejecutarse en una única transacción que hace rollback automático ante cualquier falla.
3. El equipo tiene experiencia con SQL y con el ORM elegido (Sequelize / TypeORM), lo que reduce el riesgo de errores de implementación y el tiempo de desarrollo.

La fundamentación conecta explícitamente cada razón con los motivadores del contexto. No es suficiente decir 'PostgreSQL es robusto' — hay que decir por qué esa robustez resuelve el problema específico descrito en el contexto.

4. Consecuencias

Consecuencias positivas	Trade-offs / costos
Las consultas de reportes y el flujo de compra se implementan de forma directa con SQL estándar. La consistencia transaccional está garantizada por el motor, sin lógica adicional en la aplicación. El equipo puede usar el ORM que ya conoce, acelerando el desarrollo.	Escalabilidad horizontal limitada: PostgreSQL escala mejor verticalmente. Si el volumen de datos crece significativamente, se necesitaría sharding o una migración parcial. Esquema rígido: cambios en el modelo de datos requieren migraciones explícitas, lo que agrega fricción al desarrollo iterativo. Decisión relacionada: ADR-005 (estrategia de caché) debe considerar que PostgreSQL no tiene caché integrado para lecturas de alta frecuencia.

Las consecuencias negativas son tan importantes como las positivas. Una sección de consecuencias sin trade-offs es una señal de que no se analizó la decisión con honestidad. El último punto también muestra cómo los ADRs se relacionan entre sí.

7. Checklist de entrega

Verificar que cada ADR cumple con todo lo siguiente antes de entregar:

Estructura y contenido:

- ☐ ¿El título describe la decisión en una oración, sin revelar todavía la solución?
- ☐ ¿El contexto describe el PROBLEMA y sus motivadores, no la solución?
- ☐ ¿Se consideraron al menos 2-3 alternativas reales y viables?
- ☐ ¿Cada alternativa tiene ventaja Y desventaja documentadas?
- ☐ ¿Las desventajas de las alternativas descartadas conectan con los motivadores del contexto?
- ☐ ¿La fundamentación conecta cada razón con los motivadores del contexto?
- ☐ ¿Las consecuencias incluyen trade-offs negativos, no solo beneficios?

Coherencia del conjunto:

- ☐ ¿Hay al menos un ADR por cada categoría obligatoria?
- ☐ ¿Los ADRs son coherentes entre sí? (no se contradicen)
- ☐ ¿Los ADRs son coherentes con el modelo de datos y los diagramas de arquitectura?
- ☐ ¿Cada ADR tiene un ID único y un estado?

Pregunta de autoevaluación: si alguien leyera solo los ADRs sin conocer el proyecto, ¿entendería por qué el sistema está estructurado como está? Eso es lo que busca la cátedra.

Documento de referencia para los alumnos